

Highlights

A Physics-Informed Neural Network for Option Pricing under Rough Volatility: Architecture, Dimension Scaling, and Fast Parametric Calibration via the Markovian-Lifted PDE

Aviral Sharma, Kamal Preet Singh, Vipul Sharma

- A PINN solves the $(n+1)$ -dimensional Markovian-lifted rough-Heston PDE without a computational grid.
- Solve-error grows sub-linearly in the lifted dimension (< 11 px bp to $n=32$) while grid methods become infeasible.
- Explicit error decomposition separates network accuracy from model approximation, yielding a principled rule for choosing n .
- A parametric PINN over five model parameters warm-starts calibration at $\sim 9\times$ fewer true-model evaluations.
- Four stabilising design choices are identified, implemented, and ablated.

A Physics-Informed Neural Network for Option Pricing under Rough Volatility: Architecture, Dimension Scaling, and Fast Parametric Calibration via the Markovian-Lifted PDE

Aviral Sharma^{a,*}, Kamal Preet Singh^b, Vipul Sharma^c

^a*Chandigarh University, Chandigarh, India*

^b*SGT University, Gurugram, India*

^c*Independent Researcher, India*

Abstract

Rough-volatility models capture empirical features of equity and cryptocurrency option markets that classical models cannot, but they carry a severe computational penalty: the variance is non-Markovian, and there is no finite-dimensional pricing partial differential equation (PDE) to solve. The Markovian lift recovers a PDE by approximating the fractional variance kernel with n exponentials, at the cost of turning a two-dimensional problem into an $(n+1)$ -dimensional one. We solve this high-dimensional lifted PDE with a physics-informed neural network (PINN)—a fully connected, tanh-activated multilayer perceptron trained by minimising the strong-form PDE residual at randomly sampled collocation points—and ask a sharp engineering question: does the network’s accuracy degrade as the lifted dimension grows, in the same way a finite-difference grid would? It does not. Measuring the

*Corresponding author

Email address: aviral198765@gmail.com (Aviral Sharma)

network’s solve-error against an independent lifted Monte Carlo at the same n , and separating it cleanly from the model-approximation lift-error, we find the solve-error grows only sub-linearly: from 5.4 ± 1.0 to 10.7 ± 2.0 price basis points as n climbs from 4 to 32, while the naive grid requires 50^{n+1} points and is infeasible by $n=8$. We detail four architecture and training decisions that are essential for stable convergence: a hard-constrained initial-condition ansatz, an integrated-variance baseline anchor, self-adaptive loss weighting, and a causal τ -curriculum. We also build a parametric PINN that accepts the five rough-Heston parameters as additional network inputs; while too coarse to calibrate independently, it provides a warm start that, combined with a short local true-model refinement, recovers full calibration quality at approximately nine times fewer exact-pricer evaluations. All experiments use live Bitcoin option data from the Deribit exchange, returning a genuinely rough Hurst exponent ($H \approx 0.09$) with strong negative correlation ($\rho \approx -0.79$).

Keywords: Physics-informed neural networks, rough volatility, Markovian lift, option pricing, curse of dimensionality, model calibration, deep learning, scientific machine learning

1. Introduction

Every derivative-pricing problem is, at its core, a PDE. For the classical models of quantitative finance that PDE is low-dimensional and a finite-difference grid dispatches it in milliseconds. Rough volatility breaks this comfortable picture. Motivated by the empirical roughness of realised variance [9, 4] and the steep short-maturity skew that classical stochastic-volatility models such as Heston [12] cannot reproduce, and with microstructural foun-

dations in high-frequency trading dynamics [8], rough-volatility models drive the variance with a fractional noise whose Hurst exponent $H < 1/2$. That single fact—whose lineage runs back to long-memory volatility [6]—makes the variance process neither Markovian nor a semimartingale, leaving *no* finite-dimensional pricing PDE to discretise at all [9, 7].

The standard engineering escape is the *Markovian lift*: approximate the fractional kernel by a sum of n exponentials and recover a genuine $(n+1)$ -dimensional PDE [2, 1]. The lift is an approximation whose quality improves with n —the relative L^2 kernel error falls from 25.6% at $n=5$ to 0.39% at $n=100$ —but it couples the approximation-order knob n directly to the PDE dimension. That coupling makes the lifted PDE a clean engineering stress-test for mesh-free neural solvers: as we turn n up to improve model fidelity, does the solver hold?

Physics-informed neural networks (PINNs) are built for this setting. Rather than laying down a grid, a PINN represents the price as a neural network and trains it to satisfy the PDE at randomly sampled collocation points [20]. Because nothing in the method scales with a grid, it sidesteps the exponential blow-up that kills finite differences in high dimensions, in the same way as the deep Galerkin and deep BSDE solvers [21, 11]. For pricing it brings two further engineering advantages: the physics is known exactly, so no labelled prices are needed during training; and the trained network is differentiable in its inputs, which we exploit to calibrate by gradient descent.

We develop, validate, and characterise a PINN for the lifted rough-Heston PDE and test it throughout on Bitcoin (BTC) options—a liquid, high-skew market that, as we show, calibrates to a firmly rough Hurst exponent. Our

concrete engineering contributions are as follows.

1. **A complete, reproducible PINN implementation for the lifted PDE.** We describe the architecture, training stabilisers, and collocation strategy in full detail (Section 4), identify the four design choices that are individually necessary for stable convergence, and make the full PyTorch implementation available.
2. **A dimension-scaling result.** The network’s solve-error—how well it actually solves the PDE it is handed—grows only sub-linearly as the lifted dimension climbs, staying below roughly eleven price basis points out to $n=32$, while a naive full-tensor finite-difference grid (50^{n+1} points) is already infeasible by $n=8$. We are deliberately fair about the comparison: 50^{n+1} is the naive grid; structured schemes do better but still degrade with n . The honest engineering verdict is that a competitive grid solve at $n=16\text{--}32$ is *impractical*, not impossible.
3. **An error decomposition and a practical rule for choosing n .** We split the total pricing error into the part the network controls (solve-error, measured against a lifted Monte Carlo at the same n) and the part only more factors can fix (lift-error, measured against the exact Fourier pricer). Their crossover gives a concrete, maturity-dependent rule for the factor count.
4. **A real-market calibration.** Fit to live Deribit BTC surface data, rough Heston returns a rough Hurst exponent and strong negative correlation.
5. **Fast recalibration, honestly stated.** A parametric PINN plus a warm-start hybrid recovers from-scratch calibration quality at approx-

imately nine times fewer exact-model evaluations; we are explicit about where the standalone parametric network falls short.

We are explicit about what is *not* new. Papapantoleon and Rou [19] solved the lifted rough-Heston PDE with a mesh-free network before us. Our contribution is a residual PINN formulation, a quantitative scaling-and-decomposition analysis, a real-market calibration, and the recalibration hybrid.

2. Related work

Closest prior work.. Papapantoleon and Rou [19] price European options in rough-diffusion models by recasting the pricing PDE as an energy-minimisation problem and approximating it in a time-stepping deep gradient flow, demonstrated on the lifted Heston model. Their method is the closest to ours in both target and spirit; it differs in mechanism (time-stepping variational scheme versus residual PINN) and does not study dimension scaling, error decomposition, or the recalibration workflow.

Neural solvers for rough volatility.. Jacquier and Zurič [14] cast the path-dependent pricing equations as backward SDEs and solve them with a random-feature network, which reduces training to a least-squares regression. They share the target but not the mechanism, and dimension scaling and calibration speed are not studied.

Deep calibration surrogates.. The dominant approach for fast rough-volatility calibration is the supervised surrogate of Bayer et al. [3] and Horvath et al. [13]: a network trained offline on precomputed prices or implied volatilities,

used as a fast pricing map inside the calibration loop. It is purely data-driven; the network never sees the governing equation, carries no PDE consistency, and cannot warm-start a physics-based polish the way ours can.

PINN training pathologies and remedies.. PINNs are well-known to fail when loss terms sit on different scales or when temporal causality is violated [17, 22]. The two stabilisers we use—self-adaptive homoscedastic-uncertainty weights [15, 18] and a causal τ -curriculum [23]—address these failure modes.

Foundational methods.. We build on PINNs [20] and deep PDE solvers [21, 11]; on rough volatility [9, 7] and its multifactor Markovian lift [2, 1]; and on classical transform pricing [12, 5] for our independent ground truth.

3. Problem formulation

3.1. Rough Heston and the Markovian lift

Under rough Heston the spot price S and its instantaneous variance V satisfy

$$dS_t = \sqrt{V_t} S_t dW_t^S, \quad (1)$$

$$V_t = V_0 + \int_0^t K(t-s) \lambda(\theta - V_s) ds + \int_0^t K(t-s) \nu \sqrt{V_s} dW_s^V, \quad (2)$$

where W^S, W^V are Brownian motions with correlation ρ , and $K(t) = t^{\alpha-1}/\Gamma(\alpha)$ is the fractional kernel with $\alpha = H + \frac{1}{2}$ and $H < \frac{1}{2}$ [9, 7]. The variance is a Volterra process: non-Markovian, non-semimartingale, and admitting no finite-dimensional PDE.

The Markovian lift replaces K with $K(t) \approx \sum_{i=1}^n c_i e^{-x_i t}$, introducing n Ornstein–Uhlenbeck factors U^i with $dU_t^i = -x_i U_t^i dt + \sqrt{V_t} dW_t^V$ and $V_t =$

$V_0 + \sum_i c_i U_t^i$ [2, 1]. The weights (c_i, x_i) are placed on a geometric quadrature grid [2]; the relative L^2 kernel error falls monotonically from 25.6% at $n=5$ to 0.39% at $n=100$, so raising n genuinely buys model fidelity.

3.2. The lifted pricing PDE

The discounted European call price $P(t, x, u)$, where $x = \log S$ and $u = (u_1, \dots, u_n)$ collects the factor values, satisfies the $(n+1)$ -dimensional backward Kolmogorov PDE

$$\begin{aligned} 0 = & P_t + \left(r - \frac{1}{2}V\right) P_x + \frac{1}{2}V P_{xx} + \sum_{i=1}^n [-x_i u_i + \lambda(\theta - V)] P_{u_i} \\ & + \frac{1}{2}\nu^2 V \sum_{i,j=1}^n P_{u_i u_j} + \rho\nu V \sum_{i=1}^n P_{xu_i} - rP, \end{aligned} \quad (3)$$

with terminal condition $P(T, x, u) = \max(e^x - K, 0)$. Because all factors share the single Brownian motion W^V , the full double sum over (i, j) and the $(n+1)$ -dimensional cross term both appear; the PDE dimension is exactly $n+1$. At $n=4$ this is a five-dimensional PDE; at $n=32$ it is thirty-three.

4. Network architecture and training

4.1. Baseline network

We represent the price as a fully connected multilayer perceptron (MLP). Every hidden unit uses the tanh activation, which provides smooth, high-order derivatives needed for automatic differentiation of the PDE residual; tanh is standard for PINN residual training and avoids the dead-neuron pathology of ReLU in the high-derivative setting [20]. All weight matrices are initialised with Xavier normal initialisation [10] and all biases with zeros.

The network maps $(\tau, x, u_1, \dots, u_n) \in \mathbb{R}^{2+n}$ to a scalar correction. For the single-point PINN (parameters fixed) we use width $W=64$ and depth $D=4$ hidden layers; for the parametric PINN (five model parameters as additional inputs, Section 7.4) we increase to $W=96$, $D=5$. Table 1 summarises all architectural and training hyperparameters.

4.2. Hard-constrained initial-condition ansatz

We train in time-to-maturity $\tau = T - t$ so the terminal condition becomes an initial condition at $\tau=0$. Rather than penalise it softly, we enforce it exactly through the ansatz

$$P(\tau, x, u) = P_{\text{base}}(\tau, x) + \tau N_{\theta}(\tau, x, u), \quad (4)$$

where N_{θ} is the network output. The τ prefactor forces the learned correction to vanish at $\tau=0$ automatically, so the network never fights the initial condition. Without this, a soft penalty for the terminal payoff oscillates against the PDE residual and prevents convergence.

4.3. Integrated-variance baseline anchor

The baseline P_{base} is a Black–Scholes call price. A critical design choice is what volatility it is anchored at. Anchoring at the *spot* variance V_0 leaves a systematic level bias: the factors mean-revert from V_0 toward θ over the life of the option, so the model’s actual expected variance is higher than V_0 . We anchor instead at the model’s expected *integrated* variance

$$w(\tau) = \int_0^{\tau} \mathbb{E}[V_s] ds, \quad (5)$$

computed by a short pre-flight Monte Carlo (the same simulation used for factor collocation). The baseline then uses $\sigma(\tau) = \sqrt{w(\tau)/\tau}$, which correctly captures the mean-reversion effect; the network learns only the residual stochastic-volatility correction. This single change removed a uniform ~ 120 vol bp level bias we observed with the V_0 anchor.

4.4. MC-informed factor collocation

The factor values u at a given maturity τ are not centred at zero: they mean-revert to a distribution set by the model dynamics. Sampling from a fixed symmetric box centred at zero causes the network to expend capacity on regions the dynamics never visit, and—critically—to under-cover the region governing the put-wing skew. Before training we run a short lifted Monte Carlo simulation (1,000 paths, 50 time steps) to estimate the mean and standard deviation of each factor at the target maturity; training collocation is then drawn from $\mathcal{N}(\hat{\mu}_i, \hat{\sigma}_i)$. An additional 20% of collocation points are drawn at twice the estimated standard deviation to extend tail coverage. This change cut smile RMSE by roughly 45% (from ~ 210 to ~ 114 vol bp) relative to symmetric-box sampling.

4.5. Self-adaptive loss weighting

Three loss terms must be balanced at each training step: the PDE-residual MSE $\mathcal{L}_{\text{PDE}}$, and the MSEs at the deep-OTM ($P \rightarrow 0$) and deep-ITM ($P \rightarrow S - Ke^{-r\tau}$) boundaries. Fixed-weight balancing is brittle: the ITM boundary target is $O(1)$ while the OTM target is $O(0)$, so a single lumped weight over-emphasises the ITM term. We adopt the learnable homoscedastic-uncertainty scheme of Kendall et al. [15] in the self-adaptive

spirit of McClenney and Braga-Neto [18]: each term $\mathcal{L}_k$ is assigned a learnable log-variance s_k and the objective is

$$\mathcal{L} = \sum_{k \in \{\text{PDE}, \text{OTM}, \text{ITM}\}} [e^{-s_k} \mathcal{L}_k + s_k]. \quad (6)$$

The s_k are optimised jointly with the network weights; large or noisy terms are down-weighted automatically. We never hand-tune a boundary weight.

4.6. Causal τ -curriculum

PINNs that train simultaneously across all time-to-maturities can be pulled toward spurious solutions at long τ before the short- τ region has converged, because the residual at long τ depends causally on the short- τ solution [23]. We address this with a causal curriculum: training begins at $\tau_{\max} = 0.15T$ and linearly ramps to the full T over the first half of training. The short- τ solution is therefore established before the long- τ residual can perturb it.

4.7. Training algorithm

Algorithm 1 summarises the full training procedure.

5. Evaluation methodology

5.1. Two independent ground truths

We hold the network to two references validated independently.

Fourier pricer. A semi-analytic pricer that solves the fractional Riccati equation for the rough Heston characteristic function [7] and inverts by Fourier integration [5]. It never uses the lift, so it is the ground truth of the

true model ($n \rightarrow \infty$); in the flat-volatility limit it reproduces Black–Scholes to 3.7×10^{-7} in price.

Lifted Monte Carlo. An exponential-integrator Euler scheme that integrates the n -factor lifted SDE with the $-x_i U^i$ drift handled exactly:

$$U_{t+\Delta t}^i = e^{-x_i \Delta t} U_t^i + \int_t^{t+\Delta t} e^{-x_i(t+\Delta t-s)} \sqrt{V_s} dW_s^V. \quad (7)$$

The exponential-integrator is essential: the mean reversions (x_i) span roughly ten orders of magnitude for small H , and a naive Euler scheme diverges. The lifted MC matches Black–Scholes within Monte Carlo error (1,000 paths, 50 steps) and agrees with the Fourier pricer for large n . At finite n , the lifted MC is the *exact solution of the PDE the network solves*; the Fourier pricer is the exact solution of the model.

5.2. Error decomposition

Rather than reporting a single “PINN-versus-Fourier” figure, we decompose the error per strike and in price space (where the signed contributions add exactly),

$$\underbrace{(\text{PINN} - \text{Fourier})}_{\text{total}} = \underbrace{(\text{PINN} - \text{MC})}_{\text{solve-error}} + \underbrace{(\text{MC} - \text{Fourier})}_{\text{lift-error}}. \quad (8)$$

The *solve-error* measures how accurately the network solves the PDE it was given; it is the quantity training controls. The *lift-error* is a model approximation that only increasing n can reduce; it is not the network’s responsibility. Conflating the two gives a systematically pessimistic view of network accuracy and leads to wrong engineering decisions (e.g. adding more training iterations when the right action is to increase n).

We report price RMSE in basis points of spot (px bp; $1 \text{ px bp} = 10^{-4} \times S$) where (8) is additive. Implied-vol equivalents (vol bp) are quoted where they aid interpretation. Note that the two components carry opposite signs, so their RMSE values do not add.

5.3. Homogeneity re-scaling

We train the network at fixed strike $K=1$ and read off the smile $C(1, K)$ through the call’s degree-one homogeneity:

$$C(1, K) = K C(1/K, 1), \quad (9)$$

which holds exactly because the variance dynamics are independent of the spot level. Dropping this re-scaling and comparing mismatched options inflates the apparent error to ~ 2700 vol bp; we verified this is a comparison artefact, not a modelling failure.

6. Calibration procedures and data

6.1. Data

We use end-of-day BTC option quotes from Deribit, a cryptocurrency derivatives exchange and data provider, retrieved through its public API (no API key required). After filtering for positive bid–ask spreads and non-zero open interest, the surface covers 13 maturities and 97 points spanning a moneyness range of roughly 0.6–1.7. We set $r=0$ (standard for crypto, since there is no clean carry) and infer the forward from the put–call parity of liquid near-the-money quotes. Implied volatilities are computed from mid prices with a robust Newton–Raphson Black–Scholes inverter. The calibration objective throughout is the vega-weighted implied-volatility RMSE.

6.2. Calibration methods

We compare three calibration routes head to head.

1. **Fourier-in-the-loop.** Differential evolution (60 generations, all 5 parameters) with the Fourier pricer inside the objective, followed by Nelder–Mead polish. This is our from-scratch reference.
2. **PINN-only.** Gradient descent on the model parameters using the parametric network as a differentiable pricer, with zero Fourier calls.
3. **Hybrid.** The parametric PINN’s single-shot solution provides the initial guess; a short Nelder–Mead Fourier polish then sharpens it.

7. Results

7.1. The network scales where the grid cannot

Our headline experiment trains one architecture (width 64, depth 4) at $n=4, 8, 16, 32$ on the canonical BTC calibration at $T=0.15$, over three independent seeds (Table 2, Figure 1). The solve-error against the lifted MC grows only mildly: from 5.4 ± 1.0 to 10.7 ± 2.0 px bp—a doubling over an eight-fold rise in dimension. We resist calling it flat: an early single-seed run gave that impression, and three-seed statistics corrected us. With three seeds the error bars at adjacent n overlap, so the robust statement is the two-fold growth across the full range. Wall-clock cost stays bounded throughout (6–10 minutes on CPU, per-iteration time essentially flat at 55–62ms) because the factor-Hessian computation is vectorised over the full double sum.

Compare this against a finite-difference alternative. A naive full-tensor grid on the same $(n+1)$ -dimensional PDE at a modest 50 nodes per dimension needs 50^{n+1} points: 3.1×10^8 at $n=4$, 2.0×10^{15} at $n=8$, and 1.2×10^{56} at

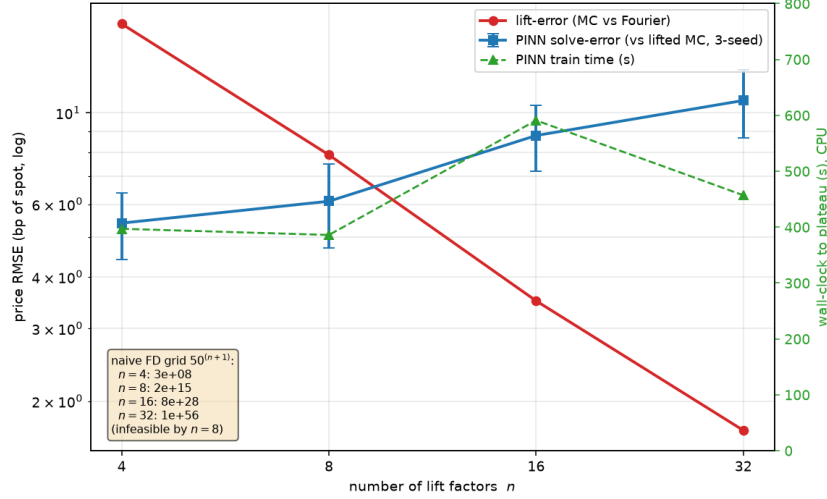


Figure 1: Dimension scaling. PINN solve-error (blue, 3-seed mean $\pm$ SD) grows sub-linearly and stays under ~ 11 px bp to $n=32$, while the lift-error (red) falls monotonically and training time (green) stays bounded. The inset shows the naive finite-difference grid size 50^{n+1} , which is infeasible by $n=8$.

$n=32$. Even $n=8$ —roughly 16 petabytes in double precision—is out of reach, while the network trains every n on a laptop. Structured schemes (sparse grids, ADI, low-rank tensor-train) reach further but still degrade with n ; none is standard or validated for this coupled ρ - ν PDE. The conservative engineering verdict is that a competitive grid solve at $n=16$ – 32 is *impractical*, not provably impossible.

7.2. Error decomposition and a design rule for n

The decomposition (8) overturned how we read our own results. A single “PINN vs Fourier” implied-vol RMSE of 114–127 vol bp at $n=4$ *looks* like a network failure. Split by (8), almost all of it is the lift: 37 vol bp of solve-error on top of 133 of lift (they partly cancel to a total of 121); with $\rho=0$

the solve-error is only 8 vol bp while the lift contributes the same 133. The network solves its PDE to well under 50 vol bp; what had appeared to be a modelling failure of the network was the finite- n model catching up to reality.

Because the lift-error shrinks monotonically with n (133, 63, 27, 12 vol bp at $n=4, 8, 16, 32$) while the solve-error grows gently, the two curves cross. At $T \approx 0.15$ the crossover falls near $n=8-10$ (Figure 1), and this gives a concrete engineering design rule: *below the crossover, add factors; above it, stop*. The crossover shifts with maturity: the lift-error grows with T (from 8.6 to 31.8 px bp at $n=4$ as T rises from 0.05 to 0.5), so longer-maturity pricing requires larger n .

Robustness sweeps over $T \in \{0.05, 0.15, 0.5\}$ and $H \in \{0.05, 0.10, 0.15\}$ confirm that the solve-error stays small and bounded everywhere (1–9.5 px bp across all configurations), while the lift-error varies predictably. The network does carry a mild directional bias at large n : at $n=32$ it systematically overprices ATM and call strikes by roughly +87 to +114 vol bp while the put wing is near zero; this bias grows with n and is the main watch item for future high- n deployments.

7.3. Bitcoin is genuinely rough

Calibrated to the full Deribit BTC surface with a full-budget optimiser (differential evolution, 60 generations, all 13 maturities, 97 points, followed by Nelder–Mead polish), rough Heston returns

$$H=0.090, \quad \rho=-0.792, \quad V_0=0.103, \quad \theta=0.253, \quad \lambda=2.567, \quad \nu=0.300,$$

at a vega-weighted implied-vol RMSE of 496 vol bp (Figure 2). The Hurst exponent is firmly rough and the correlation strongly negative—the signature

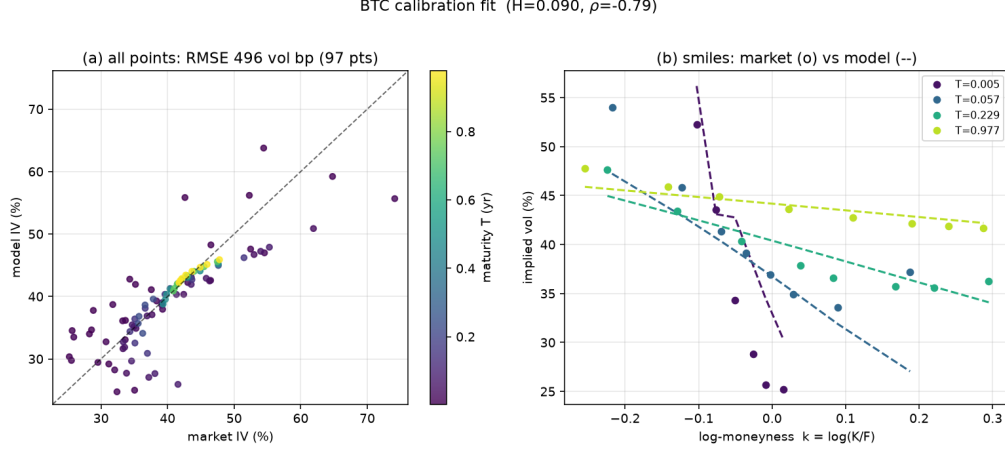


Figure 2: Rough Heston calibrated to the Deribit BTC surface (97 points, 13 maturities). (a) Market versus model implied volatility at all points (dashed: identity; colour: maturity). (b) Per-maturity smiles for four representative maturities: market (markers) versus model (dashed). The residual spread reflects a model-expressiveness limitation on wide crypto smiles, not a solver artefact.

of a crypto market with a steep, persistent left skew. We do not dress up the fit quality: at 496 volbp it is coarse, and the per-maturity smiles in Figure 2(b) show where the model strains. That coarseness is a statement about the model’s expressiveness on wide, richly curved crypto smiles, not about our solver; it is orthogonal to the solve-error studied above.

7.4. Parametric PINN and the warm-start hybrid

A single network cannot cover both the breadth of a parameter box and the accuracy of a point-trained model. Table 3 and Figure 3 report the calibration experiment with three seeds.

The parametric network adds $(H, \nu, \rho, \lambda, \theta)$ as five additional inputs, holding V_0 and the maturity fixed; we use $n=10$ (the crossover rule, Section 7.2)

and width 96, depth 5. At the box centre its solve-error is 11.0 ± 4.7 px bp versus 5.9 ± 1.2 for a single-point network at the same n —roughly a two-fold penalty. The penalty is uneven: it hovers near 9–11 px bp at typical parameter vectors but climbs to 47.7 in the rough low- H corner and 29.8 at high ν , where the PDE is stiffest.

Used as a standalone calibrator the parametric PINN is fast (approximately 2s, zero Fourier calls) but poor: 222 ± 20 vol bp under the true model. Two factors explain this. First, the network fits its own imperfect prices, not the market. Second, the single-slice BTC optimum prefers $\nu \approx 0.87$ and $\theta \approx 0.39$, which lie *outside* the training box, so the network cannot reach the true minimum. We would not recommend the standalone parametric PINN as a calibrator.

Used as a warm start it is effective. Seeding the Nelder–Mead Fourier polish with the PINN’s parameter guess reaches 29.4 ± 7.1 vol bp—on par with the from-scratch result on a 34–52% smile—in only 237 ± 46 Fourier evaluations and ≈ 23 s: roughly nine times fewer true-model calls and seven times faster than from scratch. The value proposition is amortised: train once, then recalibrate cheaply, with the exact pricer kept in the loop for a brief, decisive finish.

8. Discussion

What the scaling result does and does not say.. The finding we care about most is that the network’s own error in solving the lifted PDE degrades only gently as the dimension grows, while grid methods become impractical. We state it in its honest, weak form: the solve-error is not constant in n —a claim

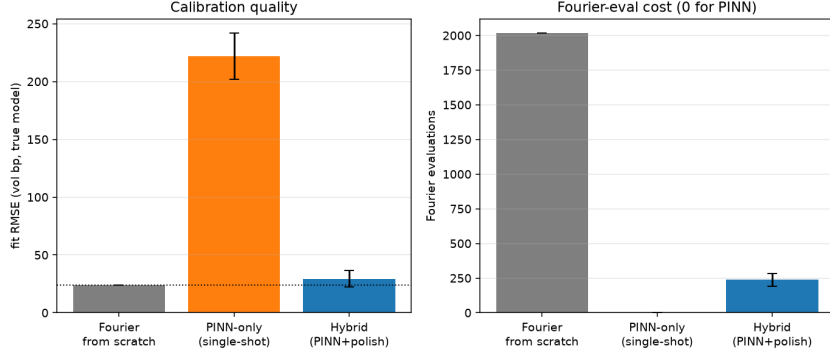


Figure 3: Hybrid calibration. Fit quality (left) and Fourier-evaluation count (right) for the three methods. The hybrid matches from-scratch quality at $\sim 9\times$ fewer Fourier evaluations; the PINN alone is fast but inaccurate.

an early single-seed run seemed to support and three-seed statistics refuted—it grows sub-linearly, roughly doubling from $n=4$ to $n=32$. And the grid comparison uses the naive 50^{n+1} count; sparse-grid, ADI, and tensor-train methods go further, so the defensible claim is *impractical*, *not impossible*. Even so, the contrast—sub-linear error and bounded wall-clock versus a grid that cannot be stored—is a concrete instance of the mesh-free advantage, and we believe it is the clearest such instance on an economically real, high-dimensional PDE.

Four design choices are load-bearing.. We ablated each stabiliser explicitly. Without the hard-constrained ansatz, the soft IC penalty oscillates against the residual and convergence stalls. Without the integrated-variance anchor, a uniform 120 vol bp level bias remains. Without MC-informed factor sampling, the smile flattens and RMSE roughly doubles. Without self-adaptive weighting or the τ -curriculum, seed variance is markedly higher (from 339 ± 62 to 310 ± 28 vol bp in an early stability comparison). Each of these design

choices is transferable to other high-dimensional PDE-pricing problems.

A lesson about validation methodology.. The error decomposition revealed something methodologically important. When a surrogate replaces an exact solver, it is dangerously easy to validate it against the wrong reference. Our network solves the finite- n lifted PDE, whose exact solution is the lifted Monte Carlo, not the $n \rightarrow \infty$ model; comparing straight to the true model conflates solve-error with lift-error. We found the two carry opposite signs and that the historical “PINN-versus-Fourier” gap was lift-dominated, not the network’s failure. *Validate against a same-fidelity reference, report the two errors separately, and let their crossover set the approximation order:* none of this is special to rough volatility, and we urge it on anyone placing a learned solver on top of an approximated operator.

Engineering trade-offs of the parametric PINN.. One network spread over five parameters is markedly less accurate than a point-trained one, worst in the rough, high-vol-of-vol corners, and calibrating with it alone is fast but unreliable. Yet the same network is a fine warm start. The value proposition is amortised: the offline training cost is paid once, and each subsequent recalibration is roughly an order of magnitude cheaper. A larger training budget would narrow the corner penalty. The key limitation today is that the training box must contain the market optimum; box design therefore requires a prior characterisation of the market regime.

Limitations and watch items.. Our evidence rests on a single market snapshot (one Deribit BTC surface), and the parametric and calibration studies fix the maturity to a single slice. Timings are single-machine CPU figures at

moderate budgets. The mild directional solve-bias at large n (ATM and call over-pricing growing to ~ 100 volbp at $n=32$) is a watch item for high- n deployments. The sub-linear trend need not continue arbitrarily far.

Future work.. The natural extensions are: (i) add the maturity as a network input for whole-surface pricing; (ii) scale width, depth, and GPU training to shrink the parametric corner penalty and probe larger n ; (iii) extend to other rough models (rough Bergomi) and to American and path-dependent payoffs; (iv) investigate whether residual reweighting or a correction term resolves the high- n directional bias; (v) benchmark the mesh-free solver directly against structured finite differences at moderate n where the latter still stand a chance.

9. Conclusion

We have presented a physics-informed neural network for the $(n+1)$ -dimensional Markovian-lifted rough-Heston pricing PDE, with a complete description of the architecture and four training stabilisers that are each necessary for stable convergence. The network’s solve-error grows only sub-linearly in the lifted dimension—staying below eleven price basis points to $n=32$ on the BTC calibration—while naive finite-difference grids become infeasible by $n=8$. Separating solve-error from lift-error is essential to read the numbers correctly and yields a concrete, maturity-dependent crossover rule for the factor count. Applied to live Bitcoin options the model is genuinely rough ($H \approx 0.09$), and a parametric PINN, while not accurate enough to calibrate independently, is an effective warm start that recalibrates at roughly nine times fewer true-model evaluations. The engineering message is

that where grid methods succumb to dimension, a mesh-free PINN degrades gracefully; its greatest practical value is as a differentiable warm start for fast, high-quality recalibration.

CRedit authorship contribution statement

Aviral Sharma: Conceptualisation, Methodology, Software, Validation, Formal analysis, Writing – original draft, Writing – review & editing. **Kamal Preet Singh:** Writing – review & editing, Funding acquisition. **Vipul Sharma:** Writing – review & editing.

Declaration of competing interests

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

The BTC option data were retrieved from the Deribit public REST API (no API key required) at <https://www.deribit.com/api/v2>. The raw data snapshot used in the experiments, the calibrated parameter file, and the complete PyTorch implementation are available at <https://github.com/aviral9876/physics-informed-nn-rough-vol>.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this manuscript the authors used Claude (Anthropic) to assist with prose drafting and structural organisation. After using this tool, the authors reviewed and edited all content as needed and take full responsibility for the accuracy and integrity of the published work. The mathematical derivations, experimental results, code, and all scientific claims are the authors' own.

Acknowledgements

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

References

- [1] E. Abi Jaber. Lifting the Heston model. *Quantitative Finance*, 19(12):1995–2013, 2019. doi: 10.1080/14697688.2019.1615113.
- [2] E. Abi Jaber and O. El Euch. Multifactor approximation of rough volatility models. *SIAM Journal on Financial Mathematics*, 10(2):309–349, 2019. doi: 10.1137/18M1170236.
- [3] C. Bayer, B. Horvath, A. Muguruza, B. Stemper, and M. Tomas. On deep calibration of (rough) stochastic volatility models, 2019. Preprint, arXiv:1908.08806.

- [4] M. Bennedsen, A. Lunde, and M. S. Pakkanen. Decoupling the short- and long-term behavior of stochastic volatility. *Journal of Financial Econometrics*, 20(5):961–1006, 2022. doi: 10.1093/jjfinec/nbaa049.
- [5] P. Carr and D. B. Madan. Option valuation using the fast Fourier transform. *Journal of Computational Finance*, 2(4):61–73, 1999. doi: 10.21314/JCF.1999.043.
- [6] F. Comte and E. Renault. Long memory in continuous-time stochastic volatility models. *Mathematical Finance*, 8(4):291–323, 1998. doi: 10.1111/1467-9965.00057.
- [7] O. El Euch and M. Rosenbaum. The characteristic function of rough Heston models. *Mathematical Finance*, 29(1):3–38, 2019. doi: 10.1111/mafi.12173.
- [8] O. El Euch, M. Fukasawa, and M. Rosenbaum. The microstructural foundations of leverage effect and rough volatility. *Finance and Stochastics*, 22(2):241–280, 2018. doi: 10.1007/s00780-018-0360-z.
- [9] J. Gatheral, T. Jaisson, and M. Rosenbaum. Volatility is rough. *Quantitative Finance*, 18(6):933–949, 2018. doi: 10.1080/14697688.2017.1393551.
- [10] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the 13th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pages 249–256, 2010.

- [11] J. Han, A. Jentzen, and W. E. Solving high-dimensional partial differential equations using deep learning. *Proceedings of the National Academy of Sciences*, 115(34):8505–8510, 2018. doi: 10.1073/pnas.1718942115.
- [12] S. L. Heston. A closed-form solution for options with stochastic volatility with applications to bond and currency options. *Review of Financial Studies*, 6(2):327–343, 1993. doi: 10.1093/rfs/6.2.327.
- [13] B. Horvath, A. Muguruza, and M. Tomas. Deep learning volatility: a deep neural network perspective on pricing and calibration in (rough) volatility models. *Quantitative Finance*, 21(1):11–27, 2021. doi: 10.1080/14697688.2020.1817974.
- [14] A. Jacquier and Ž. Zurič. Random neural networks for rough volatility, 2023. Preprint, arXiv:2305.01035.
- [15] A. Kendall, Y. Gal, and R. Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7482–7491, 2018. doi: 10.1109/CVPR.2018.00781.
- [16] D. P. Kingma and J. Ba. Adam: a method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1412.6980.
- [17] A. S. Krishnapriyan, A. Gholami, S. Zhe, R. M. Kirby, and M. W. Mahoney. Characterizing possible failure modes in physics-informed neural networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 34:26548–26560, 2021.

- [18] L. D. McClenney and U. M. Braga-Neto. Self-adaptive physics-informed neural networks. *Journal of Computational Physics*, 474:111722, 2023. doi: 10.1016/j.jcp.2022.111722.
- [19] A. Papapantoleon and J. Rou. A time-stepping deep gradient flow method for option pricing in (rough) diffusion models. *Quantitative Finance*, 25(12):2009–2020, 2025. doi: 10.1080/14697688.2025.2572318.
- [20] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.
- [21] J. Sirignano and K. Spiliopoulos. DGM: A deep learning algorithm for solving partial differential equations. *Journal of Computational Physics*, 375:1339–1364, 2018. doi: 10.1016/j.jcp.2018.08.029.
- [22] S. Wang, X. Yu, and P. Perdikaris. When and why PINNs fail to train: a neural tangent kernel perspective. *Journal of Computational Physics*, 449:110768, 2022. doi: 10.1016/j.jcp.2021.110768.
- [23] S. Wang, S. Sankaran, and P. Perdikaris. Respecting causality for training physics-informed neural networks. *Computer Methods in Applied Mechanics and Engineering*, 421:116813, 2024. doi: 10.1016/j.cma.2024.116813.

Table 1: Architecture and training hyperparameters. Values in parentheses are for the parametric PINN (Section 7.4).

Component	Value
<i>Architecture</i>	
Network type	Fully connected MLP
Hidden layers	4 (5 for parametric)
Neurons per layer	64 (96 for parametric)
Activation	tanh (all hidden layers)
Weight initialisation	Xavier normal
Bias initialisation	Zero
Input dimension	$2 + n$ for single-point; $2 + n + 5$ for parametric
Output dimension	1 (price correction N_θ)
Floating-point precision	float64 (PyTorch)
<i>Optimisation</i>	
Optimiser	Adam [16]
Initial learning rate	10^{-3}
LR schedule	Cosine annealing to 0 over all iterations
Gradient clipping	ℓ_2 norm ≤ 1.0
Training iterations	10,000–20,000 (single-point); 20,000–30,000 (parametric)
<i>Collocation sampling (per iteration)</i>	
Interior collocation	2,000 uniform + 500 near-strike (total 2,500)
Boundary points	400 (split OTM / ITM)
Log-moneyness domain	$[\log K - 1.2, \log K + 1.2]$
Factor sampling	MC-empirical mean / std (Section 4.4)
Tail coverage fraction	20% of u -points drawn at $2 \times$ std
<i>Loss weighting</i>	
	26
Adaptive scheme	Learnable log-variance (homoscedastic uncertainty)
Terms balanced	PDE residual MSE, OTM boundary MSE, ITM boundary MSE
<i>τ-curriculum</i>	
Initial learning rate	$0.15/T$

Algorithm 1 PINN training for the lifted rough-Heston PDE

Require: Model parameters $(V_0, \lambda, \theta, \nu, \rho)$; lift $(c_i, x_i)_{i=1}^n$; maturity T ; hyperparameters in Table 1

- 1: Initialise MLP N_θ (Xavier normal); learnable $s = [s_1, s_2, s_3] \leftarrow 0$
 - 2: **Pre-flight MC:** simulate lifted SDE, record $(\hat{\mu}_i, \hat{\sigma}_i)$ for each factor and term structure $w(\tau)$ $\triangleright$ Sections 4.4, 4.3
 - 3: **for** iteration $t = 1, \dots, N_{\text{iter}}$ **do**
 - 4: $\tau_{\max} \leftarrow \min(T, 0.15T + \frac{0.85T \cdot t}{0.5 N_{\text{iter}}})$ $\triangleright$ τ -curriculum, Sec. 4.6
 - 5: Sample interior: $\{(\tau_j, x_j, u_j)\}$ from $\text{Unif}([0, \tau_{\max}]) \times \text{Unif}(x_{\text{lo}}, x_{\text{hi}}) \times \mathcal{N}(\hat{\mu}, \hat{\sigma})$
 - 6: Sample boundary: $\{(\tau_j^{\text{bc}}, x_{\text{lo/hi}}, u_j^{\text{bc}})\}$
 - 7: $\hat{P} \leftarrow P_{\text{base}}(\tau, x; w) + \tau N_\theta(\tau, x, u)$ $\triangleright$ Ansatz, Eq. (4)
 - 8: $\mathcal{L}_{\text{PDE}} \leftarrow$ mean-squared PDE residual of $\hat{P}$ (Eq. (3))
 - 9: $\mathcal{L}_{\text{OTM}} \leftarrow \|\hat{P}(\tau^{\text{bc}}, x_{\text{lo}}, u^{\text{bc}})\|^2$, $\mathcal{L}_{\text{ITM}} \leftarrow \|\hat{P}(\tau^{\text{bc}}, x_{\text{hi}}, u^{\text{bc}}) - (e^{x_{\text{hi}}} - K e^{-r\tau^{\text{bc}}})\|^2$
 - 10: $\mathcal{L} \leftarrow \sum_k [e^{-s_k} \mathcal{L}_k + s_k]$ $\triangleright$ Adaptive weighting, Eq. (6)
 - 11: Backpropagate $\mathcal{L}$; clip gradients ($\ell_2 \leq 1.0$); Adam step on (N_θ, s) ; cosine LR anneal
 - 12: **end for**
-

Table 2: Dimension scaling: single-point PINN at $T=0.15$ on the canonical BTC calibration. Solve-error is 3-seed mean $\pm$ SD in price basis points of spot; lift-error and FD grid size are deterministic.

n	solve-error (px bp)	lift-error (px bp)	train time (s)	FD grid $50^{(n+1)}$
4	5.4 ± 1.0	16.4	397	3.1×10^8
8	6.1 ± 1.4	7.9	386	2.0×10^{15}
16	8.8 ± 1.6	3.5	591	7.6×10^{28}
32	10.7 ± 2.0	1.7	457	1.2×10^{56}

Table 3: Calibration of the BTC $T \approx 0.15$ slice, three methods, 3 PINN seeds. All fit-quality figures are judged by the true Fourier model over the same feasible region.

Method	Fit RMSE (vol bp)	Wall-clock (s)	Fourier evals
Fourier-loop from scratch	23.7	168	2018
PINN-only (single-shot)	222 ± 20	~ 2	0
Hybrid (PINN + Fourier polish)	29.4 ± 7.1	23	237 ± 46